HOSPITAL MANAGEMENT SYSTEM

Abstract

This report presents the design and development of a relational database system for managing hospital operations, including patient information, doctor assignments, medical records, admissions, billing, and payment processes. The objective of this project is to create a structured and efficient database model that supports accurate storage, retrieval, and management of healthcare-related information.

The database design process begins with the identification of the main entities and relationships through a conceptual Entity Relationship Diagram (ERD). The conceptual model is then transformed into a relational schema, where relationships are mapped into database tables with appropriate primary and foreign keys. The schema is normalized up to the Third Normal Form (3NF) to reduce redundancy, prevent data inconsistencies, and ensure data integrity.

The final database structure provides a foundation for implementing SQL operations, including complex queries, joins, aggregate functions, and database optimization techniques. The proposed system demonstrates how relational database principles can be applied to efficiently manage hospital data and support real-world healthcare information management requirements.

Table of Contents

Chapter1: Introduction and Requirement Analysis

Chapter 2: Database Design, ER Diagram, Relational Model, Normalization, Functional Diagram.

Chapter 3: Database Implementation (CREATE TABLES, INPUT VALUES).

Chapter 4: SQL Query Analysis (10 logical questions)

Chapter 5: Database Performance Discussion

Chapter 6: Conclusions and References

Chapter1

A) Description of the system:

The Hospital Management System (HMS) is a database-driven application designed to manage and organize the daily operations of a hospital. The system stores and manages information related to patients, doctors, appointments, departments, treatments, medical records, admissions, and billing. It helps healthcare providers maintain accurate records, improve efficiency, and provide better patient care.

B) Business Goals: The Hospital Management System aims to:

- Maintain accurate patient records.
- Improve the efficiency of appointment scheduling.
- Track patient treatments and medical history.
- Manage doctor and department information.
- Reduce paperwork and manual record-keeping.
- Generate bills and payment records accurately.
- Improve communication between departments.
- Ensure secure storage and retrieval of medical information.

C) Users of the system:

Hospital Administrators

- Manage hospital operations.
- Maintain staff and department records.
- Generate reports.

Doctors

- View patient information.
- Update diagnoses and treatments.
- Access medical records.

Nurses

- Monitor patient care.
- Update patient status and treatment information.

Receptionists

- Register patients.
- Schedule appointments.

- Manage admissions and discharges.

Billing Staff

- Generate invoices.
- Record payments.
- Manage billing information.

Chapter 2

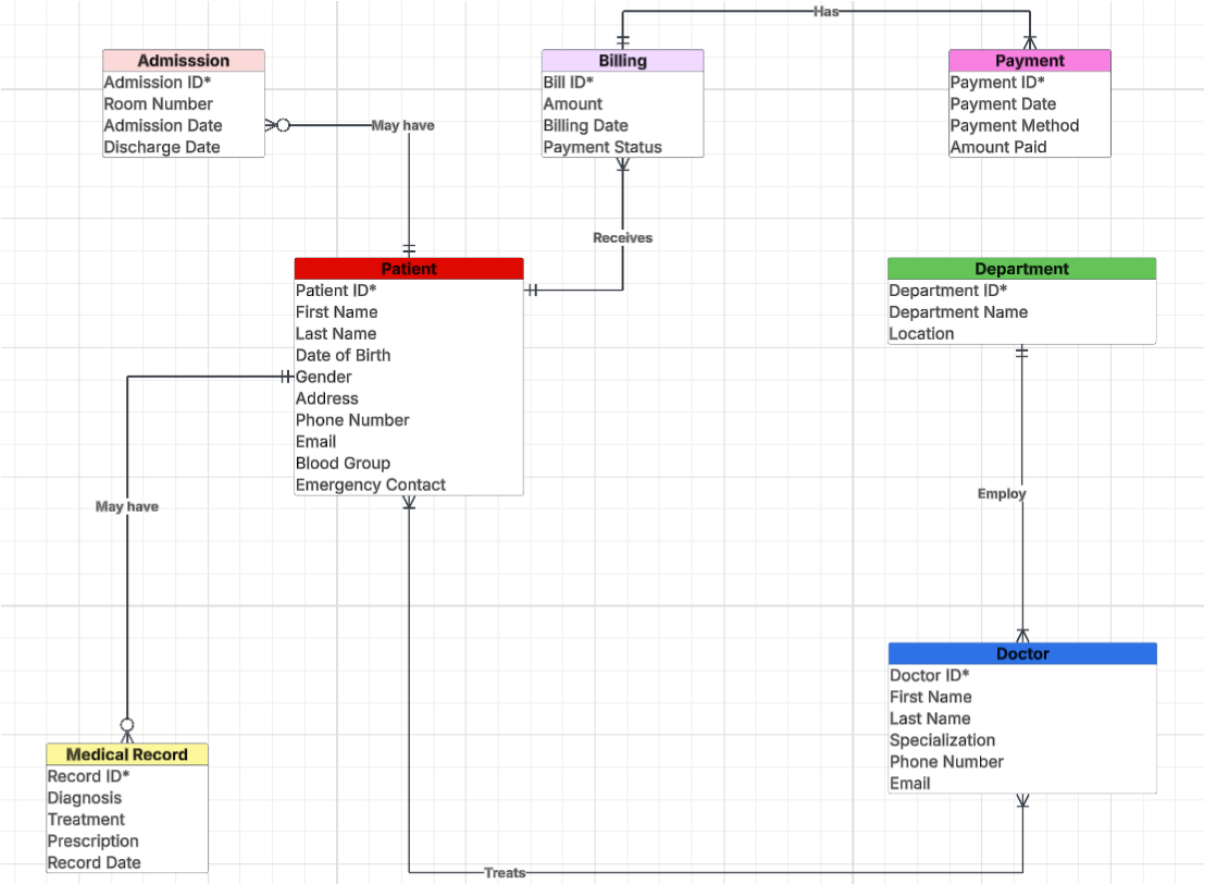
A) Data Requirements: this includes the information that will be stored in the system

- **Patient**
  (Patient ID, First Name, Last Name, Date of Birth, Gender, Address, Phone Number, Email, Blood Group, Emergency Contact)
- **Doctor**
  (Doctor ID, First Name, Last Name, Specialization, Phone Number, Email)
- **Department**
  (Department ID, Department Name, Location)
- **Medical Record**
  (Record ID, Diagnosis, Treatment, Prescription, Record Date)
- **Admission**
  (Admission ID, Room Number, Admission Date, Discharge Date)
- **Billing**
  (Bill ID, Amount, Billing Date, Payment Status)
- **Payment**
  (Payment ID, Payment Date, Payment Method, Amount Paid)

B) Relationships Between the Entities:

- A department can employ many doctors.
- Each doctor belongs to one department.
- A patient may have many medical records.
- Each medical record belongs to one and only one patient.
- A patient may have multiple hospital admissions.
- Each admission belongs to one and only one patient.
- A patient can receive one or many bills.
- Each bill belongs to one patient.
- A bill can have one or more payments.
- Each payment is associated with one bill.
- A doctor can treat many patients.
- A patient can be treated by many doctors.
- A hospital department can handle many patients through its doctors.

C) ER Diagram



D) Relational Model:

- **Patient** (Patient ID, First Name, Last Name, Date of Birth, Gender, Address, Phone Number, Email, Blood Group, Emergency Contact)
- **Appointment** (Appointment ID, Patient ID (fk), Doctor ID (fk), Appointment Date, Appointment Time, Status)
- **Doctor** (Doctor ID, First Name, Last Name, Specialization, Phone Number, Email, Department ID (fk))
- **Department** (Department ID, Department Name, Location)
- **Medical Record** (Record ID, Diagnosis, Treatment, Prescription, Record Date, Patient ID (fk))
- **Admission** (Admission ID, Room Number, Admission Date, Discharge Date, Patient ID (fk))
- **Billing** (Bill ID, Amount, Billing Date, Payment Status, Patient ID (fk))

- **Payment** (Payment ID, Payment Date, Payment Method, Amount Paid, Bill ID (fk))
- **The Appointment Table is not just a simple bridge table that resolves the M:N relationship between Patient and Doctor but it is rather an associative entity that resolves the M:N relationship and also contains attributes about the relationship**

E) Normalization:

- **First Normal Form (1NF)**; The database schema satisfies First Normal Form because all relations contain atomic values, meaning each attribute stores only a single value and there are no repeating groups.

  For example, Patient and Doctor information are stored separately, and multiple doctors assigned to patients are not stored as multiple columns. Instead, the relationship between patients and doctors is represented through the Appointment entity.

- **Second Normal Form (2NF)**; The schema satisfies Second Normal Form because all non-key attributes are fully dependent on their primary keys.

  Tables with single-attribute primary keys, such as Patient, Doctor, Department, Medical Record, Admission, Billing, and Payment, automatically satisfy 2NF because partial dependency cannot occur.

  For example, PATIENT

  Patient ID → First Name, Last Name, Date of Birth, Gender, Address, Phone Number, Email, Blood Group, Emergency Contact

  All patient attributes depend completely on Patient ID.

- **Third Normal Form (3NF)**; The schema satisfies Third Normal Form because there are no transitive dependencies, meaning non-key attributes do not depend on other non-key attributes.

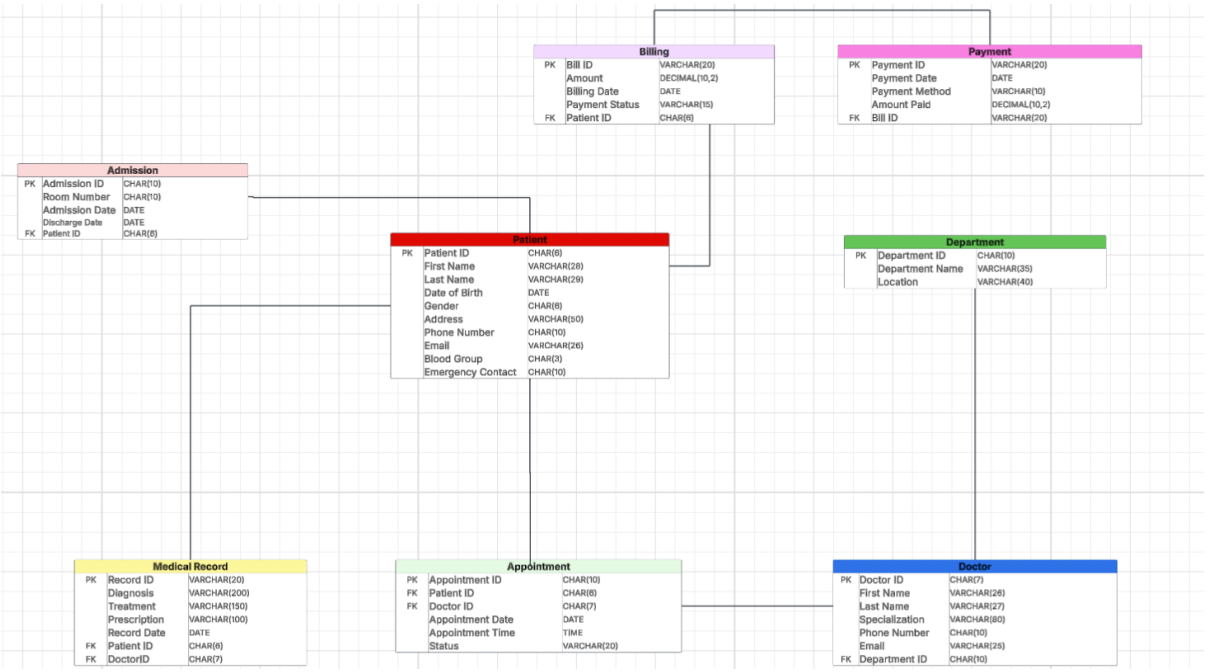
  For example, Doctor details are stored only in the Doctor table:

  DOCTOR

  Doctor ID → First Name, Last Name, Specialization, Phone Number, Email, Department ID

  and Department information is stored separately

F) Functional Diagram



Chapter 3

A) Creating Tables;

```
CREATE DATABASE Hospital_Management_System;

USE Hospital_Management_System;

CREATE TABLE Patient (
    PatientID CHAR(6) PRIMARY KEY,
    FirstName VARCHAR(28) NOT NULL,
    LastName VARCHAR(29) NOT NULL,
    DateOfBirth DATE NOT NULL,
    Gender CHAR(6),
    Address VARCHAR(50),
    PhoneNumber CHAR(10),
    Email VARCHAR(26) UNIQUE,
    BloodGroup CHAR(3),
    EmergencyContact CHAR(10)
);

CREATE TABLE Department (
    DepartmentID CHAR(10) PRIMARY KEY,
    DepartmentName VARCHAR(35) NOT NULL,
    Location VARCHAR(40)
);

CREATE TABLE Doctor (
    DoctorID CHAR(7) PRIMARY KEY,
    FirstName VARCHAR(26) NOT NULL,
    LastName VARCHAR(27) NOT NULL,
    Specialization VARCHAR(80),
    PhoneNumber CHAR(10),
    Email VARCHAR(25) UNIQUE,
    DepartmentID CHAR(10),
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);
```

```sql
CREATE TABLE Appointment (
    AppointmentID CHAR(10) PRIMARY KEY,
    PatientID CHAR(6),
    DoctorID CHAR(7),
    AppointmentDate DATE,
    AppointmentTime TIME,
    Status VARCHAR(20),
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),
    FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID)
);

CREATE TABLE MedicalRecord (
    RecordID VARCHAR(20) PRIMARY KEY,
    Diagnosis VARCHAR(200),
    Treatment VARCHAR(150),
    Prescription VARCHAR(100),
    RecordDate DATE,
    PatientID CHAR(6),
    DoctorID CHAR(7),
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),
    FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID)
);

CREATE TABLE Admission (
    AdmissionID CHAR(10) PRIMARY KEY,
    RoomNumber CHAR(10),
    AdmissionDate DATE,
    DischargeDate DATE,
    PatientID CHAR(6),
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID)
);

CREATE TABLE Billing (
     BillID VARCHAR(20) PRIMARY KEY,
     Amount DECIMAL(10,2),
     BillingDate DATE,
     PaymentStatus VARCHAR(15),
     PatientID CHAR(6),
     FOREIGN KEY (PatientID) REFERENCES Patient(PatientID)
);

CREATE TABLE Payment (
     PaymentID VARCHAR(20) PRIMARY KEY,
     PaymentDate DATE,
     PaymentMethod VARCHAR(10),
     AmountPaid DECIMAL(10,2),
     BillID VARCHAR(20),
     FOREIGN KEY (BillID) REFERENCES Billing(BillID)
);
```

B) Inputting Values; first 5 values

Patient

PatientID	FirstName	LastName	DateOfBirth	Gender	Address	PhoneNumber	Email	BloodGroup	EmergencyContact
P00001	John	Smith	1990-05-14	Male	Berlin	+494567890	john@email.com	A+	+496543210
P00002	Mary	Johnson	1985-08-22	Female	Hamburg	+494509876	mary@email.com	O+	+496501234
P00003	David	Brown	1998-01-10	Male	Munich	+495678901	david@email.com	B+	+495432109
P00004	Sophia	Miller	2000-11-05	Female	Frankfurt	+496789012	sophia@email.com	AB+	+494321098
P00005	Michael	Wilson	1993-03-18	Male	Cologne	+497890123	michael@email.com	O-	+493210987

Department

DepartmentID	DepartmentName	Location
DEP001	Cardiology	Floor 1
DEP002	Neurology	Floor 2
DEP003	Pediatrics	Floor 3
DEP004	Orthopedics	Floor 4
DEP005	Dermatology	Floor 5

Doctor

DoctorID	FirstName	LastName	Specialization	PhoneNumber	Email	DepartmentID
DOC0001	Sarah	Anderson	Cardiologist	+551111111	sarah@hospital.com	DEP001
DOC0002	James	Taylor	Neurologist	+552222222	james@hospital.com	DEP002
DOC0003	Emma	White	Pediatrician	+553333333	emma@hospital.com	DEP003
DOC0004	Robert	Clark	Orthopedic Surgeon	+554444444	robert@hospital.com	DEP004
DOC0005	Olivia	Davis	Dermatologist	+555555555	olivia@hospital.com	DEP005

Appointment

AppointmentID	PatientID	DoctorID	AppointmentDate	AppointmentTime	Status
APT0000001	P00001	DOC0001	2024-01-31	09:00:00	Completed
APT0000002	P00002	DOC0002	2024-02-01	10:30:00	Scheduled
APT0000003	P00003	DOC0003	2024-02-02	11:00:00	Completed
APT0000004	P00004	DOC0004	2024-02-03	14:00:00	Cancelled
APT0000005	P00005	DOC0005	2024-02-04	15:30:00	Scheduled

Medical Record

RecordID	Diagnosis	Treatment	Prescription	RecordDate	PatientID	DoctorID
MR001	Hypertension	Medication	Lisinopril	2024-01-31	P00001	DOC0001
MR002	Migraine	Therapy	Sumatriptan	2024-02-01	P00002	DOC0002
MR003	Common Cold	Rest and Fluids	Paracetamol	2024-02-02	P00003	DOC0003
MR004	Knee Injury	Physical Therapy	Ibuprofen	2024-02-03	P00004	DOC0004
MR005	Eczema	Skin Treatment	Hydrocortisone	2024-02-04	P00005	DOC0005

Admission

AdmissionID	RoomNumber	AdmissionDate	DischargeDate	PatientID
ADM000001	R101	2024-01-05	2024-01-09	P00001
ADM000002	R204	2024-01-06	2024-01-08	P00002
ADM000003	R305	2024-01-07	2024-01-15	P00003
ADM000004	ICU01	2024-01-10	NULL	P00004
ADM000005	R110	2024-01-11	2024-01-14	P00001

Billing

BillID	Amount	BillingDate	PaymentStatus	PatientID
BILL0000001	250.00	2024-01-05	Paid	P00001
BILL0000002	480.50	2024-01-06	Pending	P00002
BILL0000003	120.75	2024-01-07	Paid	P00003
BILL0000004	950.00	2024-01-10	Overdue	P00004
BILL0000005	300.25	2024-01-11	Paid	P00001

Payment

PaymentID	PaymentDate	PaymentMethod	AmountPaid	BillID
PAY000001	2024-01-06	Cash	250.00	BILL0000001
PAY000002	2024-01-08	Card	300.00	BILL0000002
PAY000003	2024-01-08	Bank	180.50	BILL0000002
PAY000004	2024-01-07	Insurance	120.75	BILL0000003
PAY000005	2024-01-12	PayPal	500.00	BILL0000004

Chapter 4

SQL Queries that answer 10 logical questions:

1. Display each Patient's name, appointment date, appointment status, and the doctor's name.

```
SELECT
        p.FirstName as Patients_FirstName,
        p.LastName as Patients_LastName,
        AppointmentDate,
        Status,
        d.FirstName as Doctors_FirstName,
        d.LastName as Doctors_LastName
FROM Patient as p
JOIN Appointment as a
 ON p.PatientID=a.PatientID
JOIN Doctor as d
 ON a.DoctorID=d.DoctorID;
```

2. Show the number of patients handled by each doctor.

```
SELECT
  d.DoctorID,
  d.FirstName as Doctors_FirstName,
  d.LastName as Doctors_LastName,
  COUNT(a.PatientID) as Total_patients
FROM Doctor as d
LEFT JOIN Appointment as a
  ON d.DoctorID = a.DoctorID
```

```
GROUP BY d.DoctorID, d.FirstName, d.LastName;
```

3. Classify bills as Low, Medium, or High value, using CASE Expression.

```
SELECT
        BillID,
        Amount,
        CASE
                WHEN Amount < 500 THEN 'Low'
                WHEN Amount BETWEEN 500 AND 1000 THEN
            'Medium'
                ELSE 'High'
        END AS Bill_category
FROM Billing;
```

4. Display patient names and whether their bill has been fully paid.

```
SELECT
        p.FirstName,
        p.LastName,
        Amount,
        CASE
                WHEN b.PaymentStatus = 'Paid' THEN 'Fully Paid'
                ELSE 'Outstanding'
        END AS Bill_status
FROM Patient as p
JOIN Billing as b
  ON p.PatientID = b.PatientID;
```

5. Find patients whose bill amount is above the average bill amount.

```
SELECT
  PatientID,
  Amount
FROM Billing
WHERE Amount > (
SELECT AVG(Amount)
FROM Billing
  );
```

6. Show doctors who have handled more than one appointment.

```
WITH cte_Doctor_Appointments AS
(
SELECT
DoctorID,
COUNT(*) AS Appointment_count
```

```
FROM Appointment
GROUP BY DoctorID
)
SELECT
  *
FROM cte_Doctor_Appointments
WHERE Appointment_count > 1;
```

7. Rank doctors according to the number of appointments they have handled.

```
SELECT
        d.DoctorID,
        d.FirstName,
        d.LastName,
        COUNT(a.AppointmentID) AS Total_appointments,
        RANK() OVER(ORDER BY COUNT(a.AppointmentID) DESC) AS
      Doctor_rank
FROM Doctor as d
LEFT JOIN Appointment as a
  ON d.DoctorID = a.DoctorID
GROUP BY d.DoctorID, d.FirstName, d.LastName;
```

8. Display billing records with a running total of revenue.

```
SELECT
  BillID,
  BillingDate,
  Amount,
  SUM(Amount) OVER(ORDER BY BillingDate) AS Running_revenue
FROM Billing;
```

9. Display patient name, doctor name, department, diagnosis, and treatment.

```
SELECT
        p.FirstName as Patients_FirstName,
        p.LastName as Patients_LastName,
        d.FirstName as Doctors_FirstName,
        d.LastName as Doctors_LastName,
        DepartmentName,
        Diagnosis,
        Treatment
FROM Patient as p
JOIN MedicalRecord as mr
  ON p.PatientID = mr.PatientID
```

```
JOIN Doctor as d
  ON mr.DoctorID = d.DoctorID
JOIN Department as dep
  ON d.DepartmentID = dep.DepartmentID;
```

10. Display departments that have more than one doctor.

```
SELECT
        DepartmentName,
        COUNT(d.DoctorID) AS Number_Of_Doctors
FROM Department as dep
JOIN Doctor as d
  ON dep.DepartmentID = d.DepartmentID
GROUP BY DepartmentName
HAVING COUNT(d.DoctorID) > 1;
```

Sometimes it's not enough to be able to write queries because sometimes you need access to words that are commonly used in the database or you need an automatic response when something is do in the database, for such cases we can use; **View** and **Temporary tables**, **Indexes**, **Store Procedure**, **Trigger**.

1. Create a view that displays patient appointments with doctor details.

```
CREATE VIEW PatientAppointmentView AS
SELECT
        p.FirstName as Patients_FirstName,
        p.LastName as Patients_LastName,
        AppointmentDate,
        d.FirstName AS Doctors_FirstName,
        d.LastName AS Doctors_LastName
FROM Patient as p
JOIN Appointment as a
  ON p.PatientID = a.PatientID
JOIN Doctor as d
  ON a.DoctorID = d.DoctorID;
```

To view;

```
SELECT
  *
FROM PatientAppointmentView;
```

2. Create a procedure that returns all appointments for a given patient.

```
DELIMITER //

CREATE PROCEDURE GetPatientAppointments
(
  IN PatientCode CHAR(6)
)
BEGIN

        SELECT
          *
        FROM Appointment
        WHERE PatientID = PatientCode;

END //

DELIMITER ;
```

To display what was stored in the store procedure we call it;

E.g., **CALL GetPatientAppointments('P00001');**

3. Automatically update PaymentStatus to 'Paid' when payment covers the bill amount.

```
DELIMITER //

CREATE TRIGGER trg_UpdateBillStatus
AFTER INSERT ON Payment
FOR EACH ROW
BEGIN

        UPDATE Billing
        SET PaymentStatus = 'Paid'
        WHERE BillID = NEW.BillID
         AND Amount <= NEW.AmountPaid;

END //

DELIMITER ;
```

4. Create a temporary table containing only outstanding bills.

```
CREATE TEMPORARY TABLE OutstandingBills AS
SELECT
  *
FROM Billing
WHERE PaymentStatus = 'Pending';
```

5. Improve appointment search performance by indexing frequently searched columns.

```
CREATE INDEX idx_appointment_patient
  ON Appointment(PatientID);

CREATE INDEX idx_appointment_doctor
  ON Appointment(DoctorID);
```

Chapter 5

A. Factors affecting database performance:

Several factors influence how well a relational database performs;

- **Indexing -** Without indexes, every query does a full table scan. Indexing **PatientID** and **DoctorID** on **Appointment**, **MedicalRecord, Admission**, and **Billing** dramatically speeds up joins and lookups, since these columns are used constantly in **WHERE** and **JOIN** clauses.
- **Query design** - Poorly written queries (e.g., SELECT * instead of selecting needed columns, unnecessary subqueries, missing WHERE clauses) increase I/O and processing time.
- **Normalization vs. denormalization** - The schema is normalized (separate **Doctor**, **Patient**, **Billing**, **Payment** tables linked by foreign keys), which reduces redundancy but requires joins. Over-normalization can hurt read performance, some denormalization (like views) can help.
- **Hardware resources** - CPU, RAM, and disk I/O speed (SSD vs HDD) directly affect how fast queries execute, especially for large scans or sorts.
- **Concurrency and locking** - Multiple users booking appointments or making payments simultaneously can cause lock contention on the same rows/tables.
- **Data types and table design** - Using appropriately sized data types (e.g., CHAR(7) for DoctorID instead of an oversized VARCHAR) reduces storage and speeds up comparisons.
- **Fragmentation and maintenance** - Over time, indexes and tables fragment, requiring periodic maintenance (rebuilding indexes, updating statistics).

B. How would the database perform with millions of records?

As the tables grow from tens of rows to millions (e.g., years of **Appointment** and **Billing** history across a large hospital network), several things happen;

- **Full table scans become expensive -** A query like finding all appointments for a patient without an index on **PatientID** would scan millions of rows instead of a few.
- **Indexes become essential, not optional -** The indexes you've already implemented on **PatientID**/**DoctorID** in **Appointment**, **MedicalRecord**,

and **Billing** would go from "nice to have" to "mandatory", without them, response times could go from milliseconds to seconds or minutes.

- **Joins across large tables get costlier -** Joining **Billing** and **Payment** (both potentially millions of rows) requires efficient indexing on **BillID** in both tables to avoid nested-loop scans.
- **Storage and I/O become bottlenecks -** large tables may need to be stored across multiple disk pages, increasing read latency.
- **Partitioning may become necessary -** For example, partitioning **Appointment** or **Billing** by year or by **AppointmentDate** range would let the database skip irrelevant partitions entirely when querying a specific date range.
- **Views and stored procedures still work well** but benefit from underlying indexes. A view like "patients with overdue bills" is only as fast as the indexes on the tables it queries.
- **Triggers add overhead -** A trigger that fires on every INSERT into Payment (e.g., to update PaymentStatus in Billing) adds a small cost per transaction. Negligible at small scale, but at millions of transactions it needs to be efficient.

C. Importance of **ACID** Compliance

ACID properties guarantee that database transactions are processed reliably, which is critical in a hospital system where incorrect data (like billing errors or lost medical records) can have serious consequences.

Atomicity - A transaction is all-or-nothing. Example: when a patient makes a payment, if your trigger updates **Billing.PaymentStatus** after an INSERT into Payment, atomicity ensures that if the trigger update fails, the payment insert is rolled back too. You never end up with a payment recorded but the bill still showing "Overdue," or vice versa.

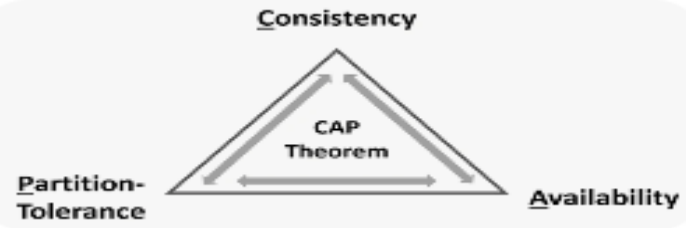
Consistency - The database moves from one valid state to another, respecting all constraints. For example: your foreign key constraints (**PatientID** in **Appointment** must exist in **Patient**, **DoctorID** must exist in **Doctor**) enforce consistency — you can never insert an appointment for a patient or doctor that doesn't exist.

Isolation - Concurrent transactions don't interfere with each other. For example: if two staff members simultaneously try to book the last available appointment slot with **DOC0001** at the same time, isolation

ensures one transaction completes fully before the other sees the updated state, preventing double-booking.

Durability - Once a transaction is committed, it stays committed, even after a system crash. For example: once a Payment record is committed for **BILL0000001**, it must survive a server crash or power failure, the hospital can't afford to lose proof that a patient paid their bill.

D. The CAP Theorem



The CAP theorem states that a distributed database can only guarantee two of the following three properties at the same time:

Consistency - Every read receives the most recent write or an error. For example: if the hospital system were distributed across multiple servers (e.g., different hospital branches), consistency would mean that once a payment is recorded at Branch A, a query at Branch B immediately reflects the updated **`PaymentStatus`**.

Availability - Every request receives a response (success or failure), even if some nodes are down. For example: if the server storing **`Billing`** data goes down, an available system would still let staff view cached **`Appointment`** data rather than returning an error.

Partition Tolerance - The system continues to operate despite network failures splitting it into multiple isolated parts. For example: if the network connection between two hospital branches drops, a partition-tolerant system keeps functioning locally at each branch rather than halting entirely.

Since network partitions are unavoidable in real distributed systems, in practice systems must choose between CP (consistency + partition tolerance, sacrificing availability during a partition) or AP (availability + partition tolerance, sacrificing strict consistency, often using "eventual consistency" instead).
It's worth noting: a traditional single-server relational database like this one (SQL with ACID guarantees) is not really a "distributed" system, so

CAP doesn't directly apply to it in practice, CAP becomes relevant only if the hospital management system was scaled out across multiple servers/data centers (e.g., replicating Patient and Billing data across regional hospital branches for redundancy). In that scenario, the hospital system would likely lean CP for billing and medical records (favoring consistency, since incorrect billing or medical data is dangerous) even at some cost to availability during a network partition, while something less critical like an appointment-reminder notification service might lean AP.

Chapter 6

Conclusion

This project involved designing and implementing a relational database for a hospital management system, covering core entities such as **Doctor**, **Patient**, **Appointment**, **MedicalRecord**, **Admission**, **Billing**, and **Payment**, along with the relationships between them. Beyond the base schema, the project extended into more advanced database features, including views for simplified data access, temporary tables for intermediate query processing, stored procedures for reusable business logic, triggers for automated data consistency, and indexes to optimize query performance on frequently searched columns like **PatientID** and **DoctorID**.

Through this process, several key database principles were reinforced. Proper normalization and foreign key constraints ensured data integrity across related tables, preventing scenarios such as appointments being booked for non-existent patients or doctors. The use of indexes demonstrated how query performance can be significantly improved, particularly as data volume grows, a consideration that becomes critical when scaling from a small test dataset to a system managing millions of records across a real hospital network. Understanding ACID properties highlighted why transactional reliability is non-negotiable in a healthcare context, where errors in billing, medical records, or appointment scheduling could have real-world consequences for patient care and hospital operations. Similarly, exploring the CAP theorem provided insight into the trade-offs a system would need to navigate if this database were ever scaled into a distributed architecture across multiple hospital branches, particularly the tension between strict consistency and continuous availability during network partitions.

Overall, this project not only produced a functional, normalized database capable of supporting core hospital operations, but also demonstrated a practical understanding of the performance, reliability, and scalability

considerations that separate a simple database from one that could realistically support a production healthcare environment. Future improvements could include implementing partitioning strategies for large historical tables, exploring read replicas to improve availability, and further optimizing stored procedures and triggers for high-concurrency scenarios as transaction volume increases.

References

- The class materials
- Claude to generate the realistic input data